



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

数据安全实验报告

Lab4: 频率隐藏 OPE 方案实现

陈语童

年级：2023 级

学号：2311887

专业：计算机科学与技术

课程教师：刘哲理

2026 年 5 月 19 日

目录

一、 实验要求	2
二、 实验原理	2
(一) OPE (保序加密)	2
(二) FH-OPE (频率隐藏的保序加密)	2
(三) UDF	3
三、 环境准备	3
(一) MySQL 的安装与配置	3
(二) Python3	5
四、 实验过程	6
(一) 编写 Server 程序	6
(二) 编译生成动态链接库	6
(三) 编写并导入 SQL 文件	6
(四) 编写 Client 程序并测试	8
(五) 更改 Client 程序并测试	10
五、 实验总结	14

一、实验要求

参照教材 6.3.3 FH-OPE 实现, 完成频率隐藏 OPE 方案的复现, 并尝试在 client.py 中修改, 完成不断插入相同数值多次的测试, 观察编码树分裂和编码更新等情况。

二、实验原理

(一) OPE (保序加密)

保留顺序加密方案 (OPE, Order-Preserving Encryption) 是指**密文保留原有明文顺序**的加密方案。当用户进行范围查询时, 仅需将范围区间的端点密文发送给云端数据库即可得到所有属于该范围的密文数据, 随后, 客户端对密文数据进行解密即可得到最终结果。

形式化的定义就是, 保序加密是一种满足:

$$x_1 < x_2 \Rightarrow \text{Enc}(x_1) < \text{Enc}(x_2)$$

的加密算法, 即明文顺序和密文顺序保持一致。

密文有序, 无需额外操作, 可以直接支持密文上的 Max、Min、Order by 等 SQL 操作。比如, 对于明文 SQL: “Select * from T where C>5 and C<10”, 可以改写为 “Select * from T where C>Enc(5) and C<Enc(10)” 即可在密文上执行范围查询。

但传统 OPE 也因密文有序, 导致其会泄露“顺序信息”。服务器虽然不知道具体值, 但知道谁大谁小, 也知道数据分布和排序结构。攻击者通过数据分布和频率分析就可能恢复大量明文, 这是传统 OPE 的安全缺陷。

(二) FH-OPE (频率隐藏的保序加密)

频率隐藏保留顺序加密 (FH-OPE, Frequency-Hiding Order-Preserving Encryption) 要求相同的明文多次加密得到的密文不同, 但密文中的顺序与明文仍然保持一致。

形式化的表达与确定性保序加密不同: 给定密文 $\text{Enc}(a)$ 和 $\text{Enc}(b)$, 如果满足 $\text{Enc}(a) < \text{Enc}(b)$ 则 $a \leq b$ 。

FH-OPE 在 OPE 基础上**隐藏频率信息**, 为了解决传统 OPE 会泄露数据分布的问题。

FH-OPE 根据客户端与服务器是否需要实时协作, 通常可以分为:

1. 交互式 FH-OPE (Interactive FH-OPE)
2. 无交互式 FH-OPE (Non-interactive FH-OPE)

有无“交互”指的是**加密时是否需要与服务器通信**

交互式 FH-OPE 通常使用二叉搜索树 (BST) 或 B 树, 由服务器维护这些树构造有序密文结构, 客户端每次加密时询问服务器应该将值插入哪里, 服务器返回前驱、后继、可用区间, 客户端再据此生成随机密文。

无交互式 FH-OPE 在客户端本地直接完成加密, 不依赖服务器。客户端看不到全局状态的情况下, 为了保证插入值既随机又保序, 无交互 FH-OPE 一般采用: “随机区间函数”而非“动态插入树”的机制。客户端加密时, 在明文的有序密文区间随机出一个加密值, 从而做到保序、隐藏频率、无需交互。客户端还可以使用伪随机函数 (PRF) 生成随机偏移: $\text{Enc}(x) = f(x) + \text{PRF}_k(r)$, 其中 $f(x)$ 保证顺序、PRF 保证随机化。

(三) UDF

用户自定义函数（User-Defined Function, UDF）是 MySQL 提供给开发者的扩展接口，使开发者不局限于数据库提供的功能，可以根据自身需求，利用 UDF 定制不同的功能函数。在 MySQL 中使用 UDF 函数，可简单分为以下几个步骤：

1. **编写 C++ 程序文件：**MySQL 为 UDF 提供了多种接口，其中至少需要实现主函数和初始化函数才可以运行，此外，还可以实现析构函数释放资源。
2. **编译生成动态链接库：**使用编译器编译的 so 文件，后续需要拷贝或链接到 MySQL 的 plugin 文件夹。
3. **在数据库中创建函数：**使用 CREATE FUNCTION 命令创建函数。
4. **调用函数：**创建完成后，即可在 SQL 语句中调用该函数，如用 SELECT 命令查询函数返回值。

三、 环境准备

本实验在 WSL2, Ubuntu 22.04 下进行。

(一) MySQL 的安装与配置

首先，使用下列命令安装 MySQL 以及开发组件：

```
sudo apt install mysql-server libmysqlclient-dev
```

安装成功后，通过下面命令以 root 身份来登录：

```
sudo mysql
```

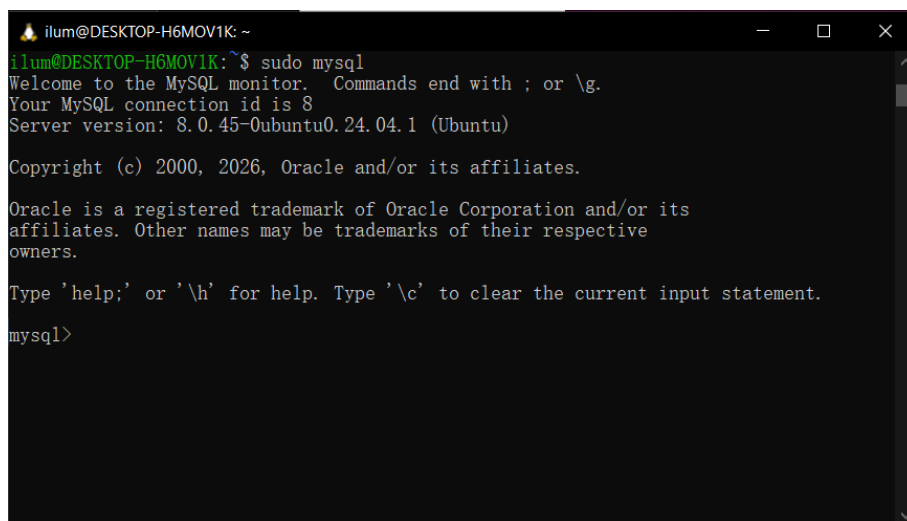
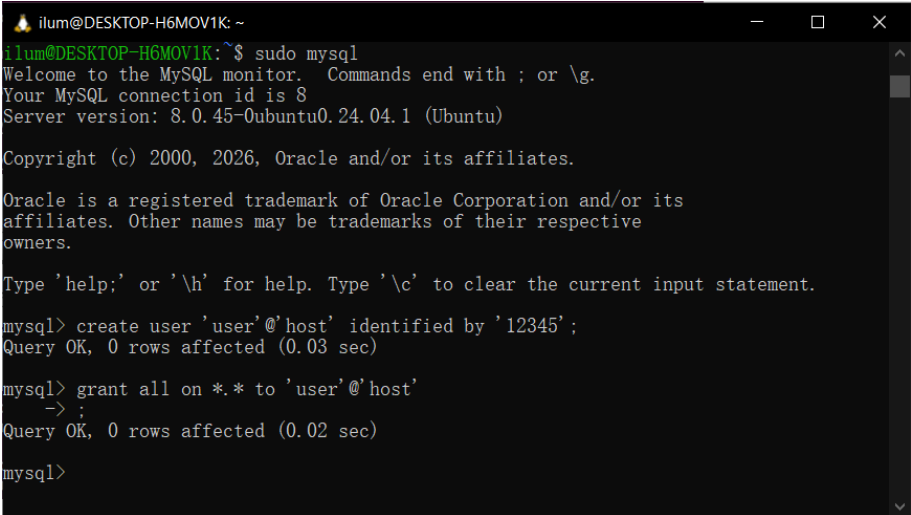


图 1：以 root 身份登录 MySQL

进入 MySQL 后，模仿下面的命令格式来创建新用户：

```
CREATE USER 'username'@'host' IDENTIFIED BY 'password';
```

创建完成后可通过 GRANT 命令授予用户权限。

A terminal window titled 'ilum@DESKTOP-H6MOV1K: ~' showing the execution of MySQL commands. The user runs 'sudo mysql' and enters the MySQL monitor. The prompt is 'mysql>'. The user enters 'create user 'user'@'host' identified by '12345';' and receives 'Query OK, 0 rows affected (0.03 sec)'. Then, the user enters 'grant all on *.* to 'user'@'host'' followed by a semicolon on the next line, and receives 'Query OK, 0 rows affected (0.02 sec)'. The prompt returns to 'mysql>'.

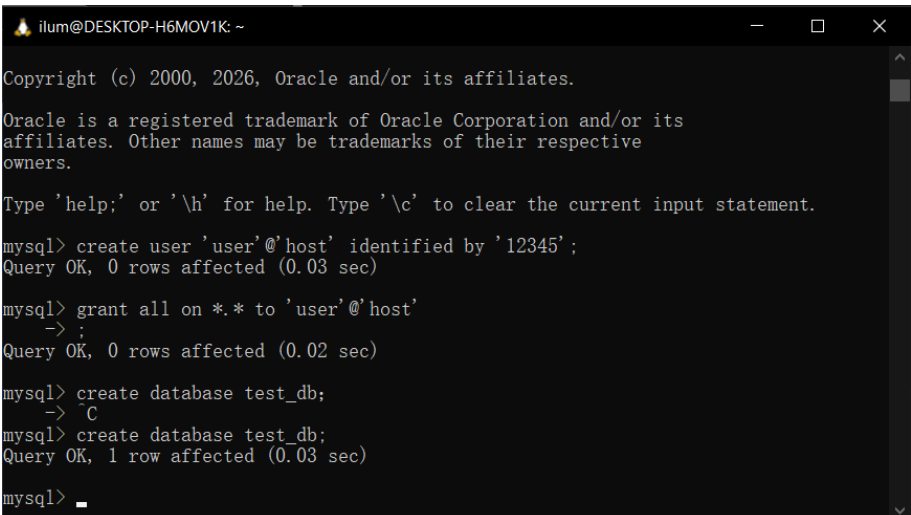
```
ilum@DESKTOP-H6MOV1K: ~  
ilum@DESKTOP-H6MOV1K:~$ sudo mysql  
Welcome to the MySQL monitor.  Commands end with ; or \g.  
Your MySQL connection id is 8  
Server version: 8.0.45-0ubuntu0.24.04.1 (Ubuntu)  
  
Copyright (c) 2000, 2026, Oracle and/or its affiliates.  
  
Oracle is a registered trademark of Oracle Corporation and/or its  
affiliates. Other names may be trademarks of their respective  
owners.  
  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
  
mysql> create user 'user'@'host' identified by '12345';  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> grant all on *.* to 'user'@'host'  
-> ;  
Query OK, 0 rows affected (0.02 sec)  
  
mysql>
```

图 2：创建用户并授权

最后用如下命令创建一个数据库：

```
CREATE database <name>;
```

将 <name> 替代为实际要创建的数据库名称即可。

A terminal window titled 'ilum@DESKTOP-H6MOV1K: ~' showing the execution of MySQL commands. The user runs 'sudo mysql' and enters the MySQL monitor. The prompt is 'mysql>'. The user enters 'create user 'user'@'host' identified by '12345';' and receives 'Query OK, 0 rows affected (0.03 sec)'. Then, the user enters 'grant all on *.* to 'user'@'host'' followed by a semicolon on the next line, and receives 'Query OK, 0 rows affected (0.02 sec)'. The prompt returns to 'mysql>'. The user then enters 'create database test_db;' followed by a semicolon on the next line, and receives 'Query OK, 1 row affected (0.03 sec)'. The prompt returns to 'mysql>'.

```
ilum@DESKTOP-H6MOV1K: ~  
ilum@DESKTOP-H6MOV1K:~$ sudo mysql  
Copyright (c) 2000, 2026, Oracle and/or its affiliates.  
  
Oracle is a registered trademark of Oracle Corporation and/or its  
affiliates. Other names may be trademarks of their respective  
owners.  
  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
  
mysql> create user 'user'@'host' identified by '12345';  
Query OK, 0 rows affected (0.03 sec)  
  
mysql> grant all on *.* to 'user'@'host'  
-> ;  
Query OK, 0 rows affected (0.02 sec)  
  
mysql> create database test_db;  
-> ;  
mysql> create database test_db;  
Query OK, 1 row affected (0.03 sec)  
  
mysql>
```

图 3：创建实验数据库

(二) Python3

已经具备 Python 环境，安装本次实验所需要的包即可。Ubuntu 系统限制 pip 安装，选择 venv 虚拟环境来安装所需包。

首先安装 venv：

```
sudo apt install python3-venv
```

安装成功后新建一个虚拟环境：

```
python3 -m venv myenv
```

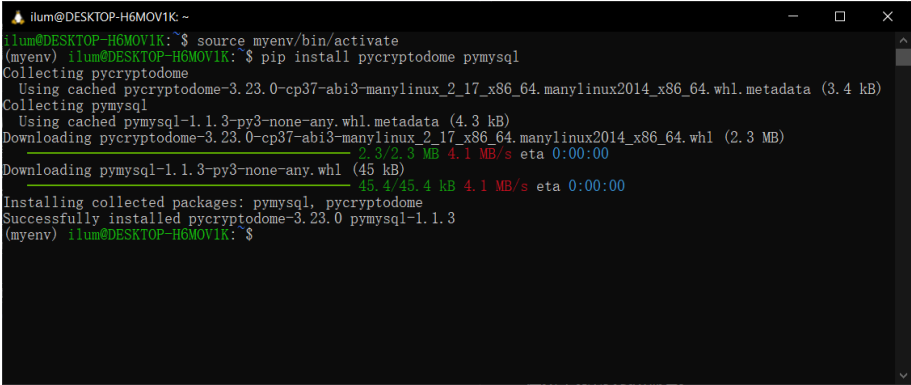
这会在用户根目录下生成一个 myenv 文件夹，也就是虚拟环境对应的目录。

接着，激活创建的虚拟环境：

```
source myenv/bin/activate
```

命令输入提示前出现 (myenv)，说明已经进入了虚拟环境。这时，再输入命令进行相关包的安装：

```
pip install pycryptodome pymysql
```



```
ilum@DESKTOP-H6MOV1K: ~  
(myenv) ilum@DESKTOP-H6MOV1K:~$ source myenv/bin/activate  
(myenv) ilum@DESKTOP-H6MOV1K:~$ pip install pycryptodome pymysql  
Collecting pycryptodome  
  Using cached pycryptodome-3.23.0-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (3.4 kB)  
Collecting pymysql  
  Using cached pymysql-1.1.3-py3-none-any.whl.metadata (4.3 kB)  
Downloading pycryptodome-3.23.0-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (2.3 MB)  
  2.3/2.3 MB 4.1 MB/s eta 0:00:00  
Downloading pymysql-1.1.3-py3-none-any.whl (45 kB)  
  45.4/45.4 kB 4.1 MB/s eta 0:00:00  
Installing collected packages: pymysql, pycryptodome  
Successfully installed pycryptodome-3.23.0 pymysql-1.1.3  
(myenv) ilum@DESKTOP-H6MOV1K:~$
```

图 4: Python 依赖安装结果

安装成功。

从而，之后的测试都需要先激活虚拟环境，再运行相应 python 程序。

四、 实验过程

(一) 编写 Server 程序

根据 FH-OPE 描述, 编写 Node.h 和 Node.cpp 文件。在教材中, 这部分代码已经完整提供, 不再展示。Node 的代码维护 FH-OPE 树结构, 用来按位置保存密文 cipher, 并给每个密文分配一个有序编码。

同样地, 教材也提供了 udf 函数的代码, 写入 UDF.cpp 即可, 代码不再展示。它把 Node.h 里的功能包装成 MySQL UDF 函数, 让 MySQL 可以调用。

(二) 编译生成动态链接库

使用如下命令编译生成动态链接库.so 文件:

```
g++ -shared -fPIC UDF.cpp Node.cpp -lcrypto -o libfhope.so
```

并将其拷贝到 MySQL 的文件夹:

```
sudo cp libope.so /usr/lib/mysql/plugin/
```

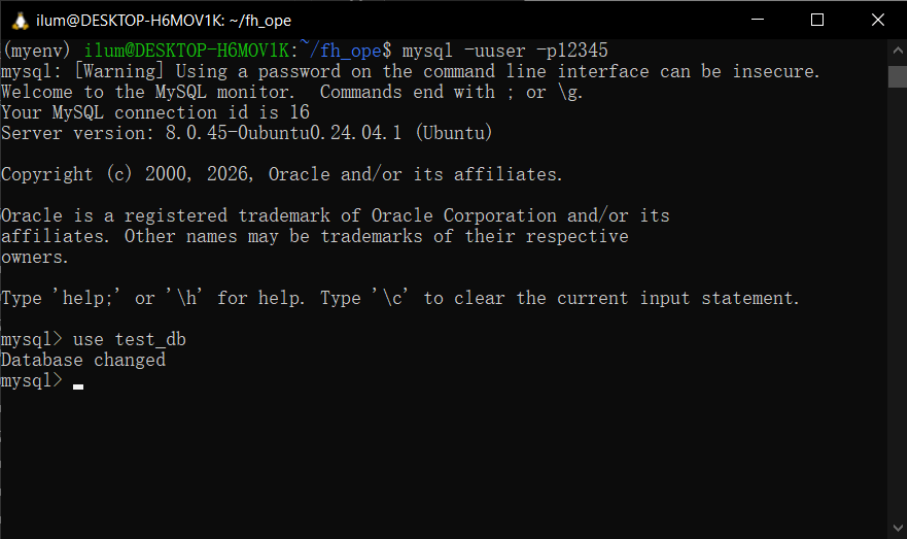
(三) 编写并导入 SQL 文件

教材给了 sql 文件内容, 复制到当前目录下 load.sql 中即可:

```
1  -- 如果原来有同名表, 则删除
2  drop table if exists example;
3  -- 创建名为example的表
4  create table example
5  (
6      encoding    bigint,
7      ciphertext  varchar(512)
8  );
9
10 -- 删除已有的函数
11 drop function if exists FHInsert;
12 drop function if exists FHSearch;
13 drop function if exists FHUpdate;
14 drop function if exists FHStart;
15 drop function if exists FHEnd;
16
17 -- 创建函数
18 create function FHInsert RETURNS INTEGER SONAME 'libfhope.so';
19 create function FHSearch RETURNS INTEGER SONAME 'libfhope.so';
20 create function FHUpdate RETURNS INTEGER SONAME 'libfhope.so';
21 create function FHStart RETURNS INTEGER SONAME 'libfhope.so';
22 create function FHEnd RETURNS INTEGER SONAME 'libfhope.so';
```

```
23
24 -- 创建插入数据的存储过程
25 drop procedure if exists pro_insert;
26 delimiter $$
27 create procedure pro_insert(IN pos int, IN ct varchar(512))
28 BEGIN
29     DECLARE i BIGINT default 0;
30     SET i = FHInsert(pos, ct);
31     insert into example values (i, ct);
32     if i = 0 then
33         -- 树结构中更新了编码，同步更新数据库中的信息
34         update example
35         set encoding = FHUpdate(ciphertext)
36         where (encoding >= FHStart() and encoding < FHEnd())
37             or (encoding = 0);
38     end if;
39 END $$
40 delimiter ;
```

而后,用之前创建的用户 user 和其密码 12345 登录 MySQL,并切换到新建的数据库 test_db:

A terminal window with a dark background. The prompt is 'ilum@DESKTOP-H6MOV1K: ~/fh_ope'. The user enters 'mysql -uuser -p12345'. The MySQL client shows a warning about using passwords on the command line, a welcome message, connection ID 16, and server version 8.0.45-0ubuntu0.24.04.1. After the copyright notice, the user enters 'use test_db' and the prompt changes to 'mysql>'.

```
ilum@DESKTOP-H6MOV1K: ~/fh_ope
(myenv) ilum@DESKTOP-H6MOV1K:~/fh_ope$ mysql -uuser -p12345
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 16
Server version: 8.0.45-0ubuntu0.24.04.1 (Ubuntu)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

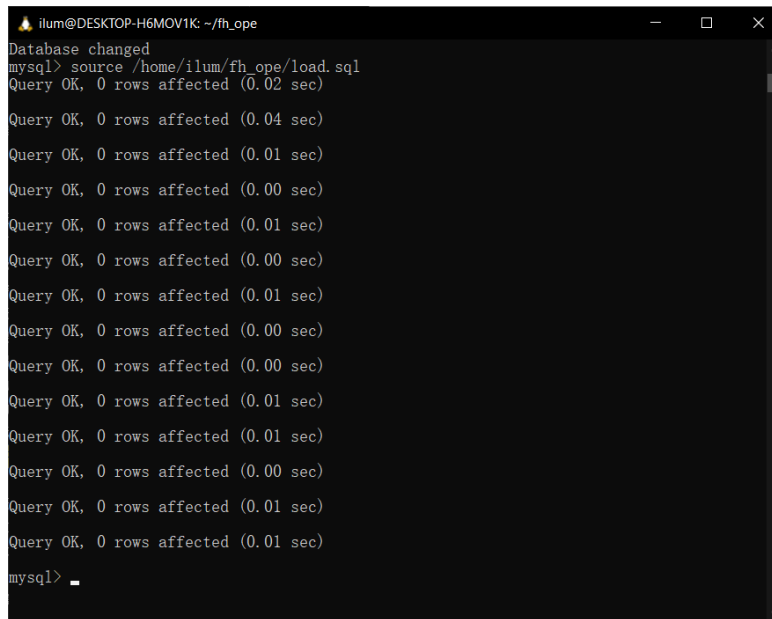
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use test_db
Database changed
mysql>
```

图 5: 登录 MySQL 并切换数据库

这时,再导入刚刚编写的 SQL 文件,使用 source + 绝对目录地址:

A terminal window with a black background and white text. The prompt is 'itum@DESKTOP-H6MOV1K: ~/fh_ope'. The user enters 'mysql> source /home/itum/fh_ope/load.sql'. The output shows 'Database changed' followed by 15 'Query OK, 0 rows affected' messages with varying execution times (e.g., 0.02 sec, 0.04 sec, 0.01 sec, 0.00 sec). The prompt returns to 'mysql>'.

```
itum@DESKTOP-H6MOV1K: ~/fh_ope
mysql> source /home/itum/fh_ope/load.sql
Database changed
Query OK, 0 rows affected (0.02 sec)
Query OK, 0 rows affected (0.04 sec)
Query OK, 0 rows affected (0.01 sec)
Query OK, 0 rows affected (0.00 sec)
Query OK, 0 rows affected (0.01 sec)
Query OK, 0 rows affected (0.00 sec)
Query OK, 0 rows affected (0.01 sec)
Query OK, 0 rows affected (0.00 sec)
Query OK, 0 rows affected (0.00 sec)
Query OK, 0 rows affected (0.00 sec)
Query OK, 0 rows affected (0.01 sec)
Query OK, 0 rows affected (0.01 sec)
Query OK, 0 rows affected (0.00 sec)
Query OK, 0 rows affected (0.01 sec)
Query OK, 0 rows affected (0.01 sec)
mysql>
```

图 6: 导入 load.sql

无报错，导入成功。

(四) 编写 Client 程序并测试

先使用教材提供的现成的 client.py 程序进行一次基础的测试：

```
1 import pymysql
2 import random
3 from Crypto.Cipher import AES
4 from Crypto.Random import get_random_bytes
5 from Crypto.Util.Padding import pad, unpad
6 import base64
7
8 local_table = {}
9 key = get_random_bytes(16)
10 base_iv = get_random_bytes(16)
11
12 def AES_ENC(plaintext, iv):
13     # AES加密
14     aes = AES.new(key, AES.MODE_CBC, iv=iv)
15     padded_data = pad(plaintext, AES.block_size, style='pkcs7')
16     ciphertext = aes.encrypt(padded_data)
17     return ciphertext
18
19 def AES_DEC(ciphertext, iv):
20     # AES解密
21     aes = AES.new(key, AES.MODE_CBC, iv=iv)
22     padded_data = aes.decrypt(ciphertext)
23     plaintext = unpad(padded_data, AES.block_size, style='pkcs7')
24     return plaintext
```

```
25
26 def Random_Encrypt(plaintext):
27     # 随机生成iv来保证加密结果的随机性
28     iv = get_random_bytes(16)
29     ciphertext = AES_ENC(iv + AES_ENC(plaintext.encode('utf-8'), iv), base_iv)
30     ciphertext = base64.b64encode(ciphertext)
31     return ciphertext.decode('utf-8')
32
33 def Random_Decrypt(ciphertext):
34     plaintext = AES_DEC(base64.b64decode(ciphertext.encode('utf-8')), base_iv)
35     plaintext = AES_DEC(plaintext[16:], plaintext[:16])
36     return plaintext.decode('utf-8')
37
38 def CalPos(plaintext):
39     # 插入plaintext, 返回对应的Pos
40     presum = sum([v for k, v in local_table.items() if k < plaintext])
41     if plaintext in local_table:
42         local_table[plaintext] += 1
43         return random.randint(presum, presum + local_table[plaintext] - 1)
44     else:
45         local_table[plaintext] = 1
46         return presum
47
48 def GetLeftPos(plaintext):
49     return sum([v for k, v in local_table.items() if k < plaintext])
50
51 def GetRightPos(plaintext):
52     return sum([v for k, v in local_table.items() if k <= plaintext])
53
54 def Insert(plaintext):
55     ciphertext = Random_Encrypt(plaintext)
56     # 连接数据库
57     conn = pymysql.connect(host='localhost', user='user', passwd='12345', database='
test_db')
58     cur = conn.cursor()
59     cur.execute(f"call pro_insert({CalPos(plaintext)}, '{ciphertext}')"
60     conn.commit()
61     conn.close()
62
63 def Search(left, right):
64     # 搜索[left,right]中的信息
65     left_pos = GetLeftPos(left)
66     right_pos = GetRightPos(right)
67     # 连接数据库
68     conn = pymysql.connect(host='localhost', user='user', passwd='12345', database='
test_db')
69     cur = conn.cursor()
70     cur.execute(
```

```

71         f"select ciphertext from example where encoding >= FHSearch({left_pos}) and
           encoding < FHSearch({right_pos})")
72     rest = cur.fetchall()
73     for x in rest:
74         print(f"ciphertext: {x[0]} plaintext: {Random_Decrypt(x[0])}")
75
76 if __name__ == '__main__':
77     # 插入明文，同时设置了一部分重复的内容
78     for ciphertext in ['apple', 'pear', 'banana', 'orange', 'cherry', 'apple', 'cherry',
79                       'orange']:
80         Insert(ciphertext)
81
82     # 假设我们搜索b和p之间的数据
83     Search('b', 'p')

```

此时运行 client.py 得到结果如下：

图 7：运行 client.py 的基础测试结果

(五) 更改 Client 程序并测试

实验要求“不断插入相同数值多次”。首先在代码开头设置重复插入的实验组参数：

```

1 # 重复插入实验组
2 TEST_REPEAT_VALUE = 'apple'
3 TEST_REPEAT_TIMES = 140
4 PRINT_EVERY = 1

```

为了能够观察到“编码树分裂和编码更新等情况”，添加了 PrintEncodingState() 函数用于打印每次插入操作的状态，包括插入的值、位置、行数、编码范围，以及更新的编码数量。具体函数格式打印方面的处理较多，不展示代码。注意其中使用到 MySQL 的 UDF 函数来获得所有编码对应的密文，以及更新的编码范围：

```

1 # 获取当前所有编码和对应的密文，按编码排序
2 def FetchEncodings():
3     conn = pymysql.connect(**DB_CONFIG)
4     cur = conn.cursor()
5     cur.execute("select encoding, ciphertext from example order by encoding, ciphertext")
6     rows = cur.fetchall()
7     conn.close()

```

```

8     return [(int(encoding), ciphertext) for encoding, ciphertext in rows]
9
10 # 获取当前编码范围和示例密文，观察更新范围和示例变化
11 def FetchUpdateRange():
12     conn = pymysql.connect(**DB_CONFIG)
13     cur = conn.cursor()
14     cur.execute("select FHStart(), FHEnd()")
15     start_update, end_update = cur.fetchone()
16     conn.close()
17     return int(start_update), int(end_update)

```

为了便于观察节点分裂，设置了如下参数便于打印相应时间节点的状态：

```

1 # Node.h 定义 M = 128. 当叶子节点的行数达到 M 时会发生分裂
2 # 为观察分裂过程，设置 SPLIT_THRESHOLD = 128，插入时行数达到该值时打印当前状态
3 SPLIT_THRESHOLD = 128

```

为了稳定制造“重复插入同一位置”的压力，使得重复值始终插入到相同值区间左端：

```

1 # 让重复值总是插入到左边，观察分裂时重复值如何分布在新旧节点中
2 FORCE_LEFT_DUPLICATES = True
3
4 .....
5
6 def CalPos(plaintext):
7     # 插入plaintext，返回对应的pos
8     presum = sum([v for k, v in local_table.items() if k < plaintext])
9     if plaintext in local_table:
10         local_table[plaintext] += 1
11         # 强制重复值插入到左边
12         if FORCE_LEFT_DUPLICATES:
13             return presum
14         return random.randint(presum, presum + local_table[plaintext] - 1)
15     else:
16         local_table[plaintext] = 1
17         return presum

```

最终，在主函数中使用 for 循环重复插入设置的值，并按照设置的时间节点打印插入状态：

```

1 if __name__ == '__main__':
2     previous = {}
3     for step in range(1, TEST_REPEAT_TIMES + 1):
4         # 插入重复值，获取其位置，并打印当前状态
5         pos, _ = Insert(TEST_REPEAT_VALUE)
6         rows = FetchEncodings()
7         # 只有在特定步骤或达到分裂点时才打印状态
8         should_print = (
9             step <= 5
10             or step % PRINT_EVERY == 0
11             or step in (SPLIT_THRESHOLD - 1, SPLIT_THRESHOLD, SPLIT_THRESHOLD + 1)
12         )

```

```
13     if should_print:
14         previous = PrintEncodingState(step, pos, rows, previous)
15     else:
16         previous = {ciphertext: encoding for encoding, ciphertext in rows}
```

新编写的 Client 程序保存为 client_new.py。

测试时, 先重新登录 MySQL, 重新导入一次 load.sql, 再用 python 命令运行 client_new.py, 测试输出结果为同文件夹下 res.txt 中文本。

输出结果的两条 entry 例子如下所示:

```
insert #001: value='apple', pos=0, rows=1
update range: [-1, -1)
ordered encodings: [2305843009213693952]
encoding updated for 0 existing row(s)

.....

insert #063: value='apple', pos=0, rows=63
update range: [0, 4611686018427387904)
ordered encodings: [0, 73201365371863296, 146402730743726592, 219604096115589888,
292805461487453184, 366006826859316480, 439208192231179776, 512409557603043072,
'...', 4465283287683661056, 4538484653055524352]
encoding updated for 62 existing row(s):
2yIkV3Eir1jK... 1 -> 73201365371863296
NVP24+e9Reiz... 2 -> 146402730743726592
... 60 more
```

其中每条 entry 的第一行说明了这条插入的编号、插入的值、插入的位置 (由于设置了始终插入到相同值区间左端, 所有 entry 的 pos 都是 0)、数据库表当前总行数。

下一行表示通过 UDF 暴露出的编码更新区间 (即 FHStart() 和 FHEnd()), 其中 [-1, -1) 表示本次没有触发重编码; [0, ...) 则表示触发了重编码。

再下一行是当前表中按 encoding 排序后的编码预览。

最后一部分展现和上一次插入后相比, 有多少条已有记录的编码发生变化, 并列举按编码顺序最先的两个变化例。

观察到结果输出中, 前 62 次都没有编码更新。这说明每次插入都插在最左边, 编码不断取左边界 0 和当前最小编码之间的中点, 所以最小编码依次变成... 4, 2, 1。第 62 次结束后:

```
insert #062: value='apple', pos=0, rows=62
update range: [-1, -1)
ordered encodings: [1, 2, 4, 8, 16, 32, 64, 128, '...', 1152921504606846976,
2305843009213693952]
encoding updated for 0 existing row(s)
```

编码更新

编码已经缩小到 1。所以，到第 63 次时，左边界 0 和最小编码 1 之间已经没有可用整数编码，于是触发 Recode()（在用于管理树的 Node.cpp 中定义）。所以第 63 次插入开始都会出现编码的更新。

此后，从第 63 次到第 128 次，更新区间一直是 [0, 4611686018427387904)，说明分裂前只有一个叶子节点，重编码会影响整个叶子节点中的记录。

编码树分裂

上面的过程一直进行到第 128 次，这时叶子节点大小为 128，达到 Node.h 中预设的上限，必须分裂。这时候输出可以观察到，检测到了 LeafNode::rebalance() 的执行，发生了**叶子节点分裂**：

```
insert #128: value='apple', pos=0, rows=128
update range: [0, 4611686018427387904)
ordered encodings: [0, 36028797018963968, 72057594037927936, 108086391056891904,
144115188075855872, 180143985094819840, 216172782113783808, 252201579132747776,
'...', 4539628424389459968, 4575657221408423936]
encoding updated for 127 existing row(s):
15AC1+l0tS2d... 0 -> 36028797018963968
PzBv4fPppfGz... 36312488334073920 -> 72057594037927936
... 125 more
leaf split point reached: rows == M == 128

insert #129: value='apple', pos=0, rows=129
update range: [0, 2305843009213693952)
ordered encodings: [0, 35474507834056832, 70949015668113664, 106423523502170496,
141898031336227328, 177372539170284160, 212847047004340992, 248321554838397824,
'...', 4539628424389459968, 4575657221408423936]
encoding updated for 64 existing row(s):
T1zk+Nk9Xgju... 0 -> 35474507834056832
15AC1+l0tS2d... 36028797018963968 -> 70949015668113664
... 62 more
```

可以看到，从 129 次插入开始，更新区间从原先的 [0, 4611686018427387904) 改变为 [0, 2305843009213693952)，缩小到了一半。同时，更新行数从第 128 次的 127 条降到第 129 次的 64 条——均可说明原来的大叶子已经分裂成左右两个叶子，后续插入都落在左叶子里，因此后续只更新左叶子的编码，右叶子不再受影响。

发生编码树分裂的机理是，Node.h 中定义了 `const int M = 128;`，并且在 Node.cpp 中实现 `insert_node()` 函数时有：

```
1 .....
2     if (this->child.size() >= M)
3     {
4         this->rebalance();
5     }
```

所以当叶子节点大小到达设定的 M 这一上限时就会触发 `rebalance()` 进行编码树分裂。

五、 实验总结

本实验在 MySQL 的客户端插入查询任务上复现了频率隐藏 OPE 方案的基本流程，期间经历了 MySQL 环境配置、UDF 函数导入、动态链接库编译等准备工作。基本复现成功后，通过不断插入相同明文值的实验，观察到 FH-OPE 中相同明文会被加密为不同密文，同时仍然保持可用于范围查询的有序编码关系。本次实验让我对于 FH-OPE 如何在保序查询能力和频率隐藏之间取得平衡有了更直观的理解，也进一步熟悉了 MySQL UDF 与 Python 客户端协同实现数据库加密的过程。

本次实验的代码已经上传至 GitHub 仓库：[data-security/lab4](https://github.com/data-security/lab4).